

Reviewer Pack — Minimal Geometric Flows and Resolution Proof Width

Entry 224e986ab71c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 08:04:59 UTC

Minimal Geometric Flows and Resolution Proof Width

Entry ID: 224e986ab71c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 08:04:59 UTC

1. Conjecture

Field A (mathematical branch): Geometric flows in differential geometry **Field B** (complexity object): Resolution proof complexity

Statement:

The minimal amount of time (in discrete time steps) required for a geometric flow to separate two points in the resolution proof space is linearly correlated with the resolution proof width, such that the ratio of the minimal flow time to the resolution proof width is $\Theta(1)$.

Rationale (proposer's reasoning):

Geometric flows, like the heat equation or Ricci flow, have been used to study the convergence of solutions in geometry. The resolution proof space can be thought of as a complex structure, where the resolution process acts as a flow. If there exists a linear correlation between the geometric flow time and the resolution proof width, it could potentially reveal underlying structures that are not evident from direct analysis of resolution proofs.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ffdada20aff4e35a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the generated Tseitin formulas with sizes $n \leq 40$, the ratio of the minimal flow time to the resolution proof width falls within a range $[0.5, 2]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric flows AND resolution proof complexity - resolution proof width IN GEOMETRIC FLOWS - minimal geometric flows AND proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate OR clauses
        for i in range(1, n+1):
            clause = [f'x{i}', f'y{i}']
            clauses.append(clause)

        # Generate AND clauses
        for i in range(2, n+1):
            clause = [f'x{i-1}', f'y{i}', f'z{i}']
            clauses.append(clause)

        # Generate final OR clause
        final_clause = [f'z{n}', 'F']
        clauses.append(final_clause)

    return variables, clauses

def resolution(clauses):
    while True:
        new_clauses = []
        found_resolvent = False

        for i in range(len(clauses)):
            for j in range(i+1, len(clauses)):
                clause_i = set(clauses[i])
                clause_j = set(clauses[j])
```

```

        # Find complementary literals
        common_literals = [lit for lit in clause_i if -lit in clause_j]
        if common_literals:
            resolvent = (clause_i - {common_literals[0]}) | (clause_j - {-common_literals[0]})
            new_clauses.append(list(resolvent))
            found_resolvent = True

    if not found_resolvent:
        return len(clauses)

    # Remove duplicates and empty clauses
    clauses = list({tuple(sorted(c)) for c in new_clauses if c})

def geometric_flow_time(n):
    # Simulate a simple geometric flow that separates points by flipping variables
    steps = 0
    while True:
        steps += 1
        for i in range(1, n+1):
            if random.choice([True, False]):
                return steps

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        variables, clauses = generate_tseitin(n)
        w_phi_G = resolution(clauses)
        flow_time = geometric_flow_time(n)

        if flow_time == 0 or w_phi_G == 0:
            continue

        ratio = flow_time / w_phi_G
        total_metric_value += ratio
        instances_tested += 1
        n_max = max(n_max, n)

        if not (0.5 <= ratio <= 2):
            conjecture_holds = False
            counterexample = f"n={n}, flow_time={flow_time}, w_phi_G={w_phi_G}"

metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0

return {
    "metric_name": "Flow Time to Resolution Width Ratio",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
}

```

```

        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4630eb8.py", line 115, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4630eb8.py", line 83, in run_trial
    w_phi_G = resolution(clauses)
                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a4630eb8.py", line 52, in resolution
    common_literals = [lit for lit in clause_i if -lit in clause_j]
                        ~~~~~^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a `TypeError`, which prevented the generation of data necessary to evaluate the conjecture. | next: Review and fix the error in the test code to ensure it can complete without crashing and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19310
2	propose	ollama_remote	glm4:latest	0	0	13379
3	preregistration	ollama_remote	glm4:latest	0	0	16609
4	novelty	ollama_remote	glm4:latest	0	0	14428
5	novelty	ollama_remote	glm4:latest	0	0	12559
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11577
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17527
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22973
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12533
10	judge	ollama_remote	glm4:latest	0	0	8694

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 149589 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/224e986ab71c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/224e986ab71c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/224e986ab71c.tar.gz` (if generated)